Recent Advances in Deep Learning - Spring 2026

Final Exam Answer Key and Grading Rubric

Total Marks: 100

Part I: True/False Questions (10 questions, 1 mark each, Total: 10 marks)

1. ✓ (The chain rule is the cornerstone of backpropagation.)
2. × (The Adam optimizer uses both the first moment estimation and second moment estimation of the gradient, combining the ideas of Momentum and RMSProp.)
3. ✓ (A too large dot product pushes the Softmax function output towards a one-hot vector, entering a saturated region with extremely small gradients; dividing by $\sqrt{d_k}$ smooths the distribution.)
4. × (Infinitely increasing depth and width leads to severe overfitting. Performance on the training set may improve, but performance on the test set will degrade.)
5. ✓ (If the input to ReLU is consistently less than 0, the gradient is 0, and weights cannot be updated, leading to "Dying ReLU" neurons.)
6. ✓ (The core claim of the Transformer paper is "Attention is All You Need", completely abandoning traditional recurrent and convolutional operations.)
7. × (Stochastic Gradient Descent (SGD) uses only a single sample or a very small Mini-batch to calculate the gradient per iteration. Using the entire training set is standard Batch Gradient Descent (BGD).)
8. × (Calculating correlation scores is the job of the Query and Key matrices. The Value matrix is the actual information carrier that is weighted and summed based on the calculated weights.)
9. ✓ (Skip connections allow gradients to flow directly to shallower layers through identity mappings, greatly alleviating vanishing gradients.)
10. ✓ (Layer Norm indeed normalizes a single sample by calculating the mean and variance across all its channel/feature dimensions, unlike Batch Norm.)

Part II: Multiple Choice Questions (20 questions, 2 marks each, Total: 40 marks)

1. **C.** $N \times M$ (Assuming the input is a $1 \times N$ row vector and the weight matrix is $N \times M$, the output is $1 \times M$.)
2. **D. Adam** (Adam combines both adaptive learning rates and a momentum mechanism.)
3. **B. Softmax** $\left(\frac{QK^T}{\sqrt{d_k}}\right)V$
4. **B. Vanishing gradients**
5. **C. To prevent model overfitting**
6. **C. ReLU and its variants**
7. **B. Its computational complexity grows exponentially compared to single-head attention.** (The total dimension of multi-head attention is usually consistent with single-head attention. By dividing dimensions and computing in parallel, the complexity remains roughly on the same order of magnitude.)
8. **B.** $O(L^2 \cdot D)$ ($Q \times K^T$ produces an $L \times L$ matrix, consuming $O(L^2 D)$; multiplying by V consumes another $O(L^2 D)$.)
9. **B. Simulating the inertial motion of an object under friction, reducing update oscillations.**
10. **C. Softmax, Cross-Entropy Loss**
11. **B. Positional Encoding**
12. **B. Layer Normalization**
13. **B. 2, with an activation function in between.** (Typically Linear → ReLU/GELU → Linear)
14. **C. Decreasing Batch Size.** (Decreasing Batch Size mainly affects gradient noise and memory footprint; it is not a targeted method to prevent overfitting. A, B, and D are standard anti-overfitting techniques.)
15. **A. It causes model weights to tend towards a sparse matrix (many weights become 0).**
16. **B. By multiplying the output of the previous layer by a learnable weight matrix** W^Q**.**
17. **C. Warmup with Cosine Decay**
18. **B. Masked Self-Attention** (Masks out future information to ensure causality during autoregressive generation.)
19. **B. Every differentiable operation can be represented as a node on the computational graph, and backpropagation is executing the chain rule on the graph.**
20. **B. Splitting the image into multiple non-overlapping Patches and flattening them into a sequence.**

Part III: Short Answer & Essay Questions (Open-end, 5 questions, 10 marks each, Total: 50 marks)

1. **Multilayer Perceptron and Backpropagation (10 marks)**

   Derivation:
   Let the intermediate variable be $z = W_1 x + b_1$, then the hidden layer output is $h = \sigma(z)$.
   The loss function is $L = \frac{1}{2}(y-t)^T(y-t)$.
   According to the Chain Rule, we differentiate from the output layer to the input layer:

   - Partial derivative of loss w.r.t output y: $\frac{\partial L}{\partial y} = y - t$ (2 marks)
   - Partial derivative of loss w.r.t hidden layer output h: $\frac{\partial L}{\partial h} = W_2^T \frac{\partial L}{\partial y} = W_2^T(y-t)$ (2 marks)
   - Partial derivative of loss w.r.t pre-activation variable z (introducing Hadamard product $\odot$):
     $\frac{\partial L}{\partial z} = \frac{\partial L}{\partial h} \odot \sigma'(z) = (W_2^T(y-t)) \odot \sigma'(W_1 x + b_1)$ (3 marks)
   - Partial derivative of loss w.r.t weight matrix W_1:
     $\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial z} x^T = \left[(W_2^T(y-t)) \odot \sigma'(W_1 x + b_1)\right] x^T$ (3 marks)

2. **Analysis of Neural Network Optimizers (10 marks)**

   Key Points:

   - **Core Idea (4 marks):** Adam (Adaptive Moment Estimation) combines two optimization strategies. First, it keeps an exponentially weighted moving average of the gradient (first moment, Momentum) to maintain a consistent direction in noisy gradients. Second, it keeps an exponentially weighted moving average of the squared gradients (second moment, RMSProp concept) to calculate adaptive, independent learning rates for different parameters (slowing down updates for parameters with large gradients, and speeding up updates for parameters with small gradients).
   - **Bias Correction (2 marks):** It includes a bias correction mechanism for when the first and second moments are initialized to 0, which is especially effective in the early stages of training.
   - **Why Sometimes Choose SGD+Momentum (4 marks):** Although Adam converges quickly, in certain tasks (especially image classification in computer vision), Adam is prone to converging into "sharp" local minima, leading to decreased generalization ability. Conversely, SGD with momentum (combined with a good learning rate decay strategy), despite taking longer to train, is more likely to converge to "flat" minima, thereby achieving better generalization performance on the test set (closing the Generalization Gap).

3. **Detailed Explanation of Attention Mechanism Computation (10 marks)**

   Key Points:

   - **(1) Generation of Q, K, V (3 marks):** By multiplying the input sequence matrix X by three independent learnable weight matrices W^Q, W^K, W^V respectively: $Q = XW^Q, K = XW^K, V = XW^V$.
   - **(2) Reason for Dividing by $\sqrt{d_k}$ (4 marks):** When the feature dimension d_k is large, the variance of the dot product distribution of Q and K^T becomes extremely large, causing some elements in the vector to have an absolute advantage in magnitude. After

passing through the Softmax function, most gradient values will approach 0 (entering the saturated region), causing vanishing gradients during backpropagation. Dividing by $\sqrt{d_k}$ (the square root of the variance) rescales the dot product variance back to 1, ensuring stable training.

- **(3) Advantages of the Multi-Head Mechanism (3 marks):** It allows the model to jointly attend to information from different representation subspaces at different positions in parallel (e.g., one head focuses on syntactic structure, another on semantic correspondence). If using only a single head, this richness of information might be diluted by averaging.

4. **Architecture Design and Gradient Propagation Analysis (10 marks)**

   Key Points:

   - **Mathematical Essence of Vanishing Gradients (5 marks):** In deep networks, according to the chain rule, the total gradient is the continuous product of gradients from each layer (usually derivatives of activation functions and weight matrices). If the derivative of the activation function (e.g., Sigmoid's maximum derivative is only 0.25) or the initialized weight values remain less than 1, multiplying many values less than 1 results in exponential decay. When propagating back to shallow layers, the gradient becomes almost 0, preventing the weights of shallow layers from updating.
   - **ResNet's Mitigation Mechanism (5 marks):** Residual networks introduce Skip Connections, rewriting the layer's output as $H(x) = F(x) + x$. During backpropagation differentiation, $\frac{\partial H}{\partial x} = \frac{\partial F}{\partial x} + 1$. Because of this extra "+1" term, even if $\frac{\partial F}{\partial x}$ approaches 0 due to network depth, the gradient can still flow back to previous layers completely unattenuated through this "+1" identity mapping path, fundamentally preventing the gradient from vanishing during backpropagation.

5. **Recent Advances and Bottlenecks in Deep Learning (10 marks)**

   Key Points:

   - **Advantages Over LSTM (5 marks):**
     (a) **Parallelization Capability:** LSTM computations have time dependency (computation at time t must wait for time $t-1$ to finish), making parallelization across the sequence dimension impossible during training. The Transformer's Self-Attention can compute all tokens in the sequence simultaneously, vastly improving hardware utilization and training speed.
     (b) **Long-Range Dependencies:** When LSTMs process long sequences, information decays over steps, leading to forgetting. In Transformers, the distance between any two tokens is $O(1)$, and its global receptive field perfectly extracts long-range dependency information.
   - **Core Computational Bottleneck (3 marks):** Both the computational time complexity and memory space complexity of standard self-attention are $O(L^2)$ (where L is the sequence length). When processing 100,000-level tokens, the L^2 term causes Out of Memory (OOM) errors.
   - **Academic Approaches to Mitigating this Bottleneck (2 marks, list and briefly describe any one):**
     (a) **Sparse Attention:** E.g., Longformer/BigBird. They replace the full $N \times N$ attention matrix with sliding windows, local attention, and a few global attention tokens, reducing complexity to $O(L)$.

(b) **Linear Attention:** E.g., Linformer, Performer. They alter the associativity of multiplication using kernel methods or low-rank factorization, dropping computational complexity to $O(L)$.

(c) **FlashAttention:** Uses low-level IO-aware optimization to perform tiling and recomputation between SRAM and HBM. Although it does not change the mathematical complexity, it drastically accelerates the actual computation speed of long sequences on GPUs and saves memory.